

It's also a nightmare for understanding what's going on.

Which is exactly where observability comes in.

What we mean when we say observability

The word gets used loosely. Vendors love it. Marketing teams love it. The actual definition, borrowed from control theory, is the ability to understand a system's internal state by looking at its external outputs.

That's the formal version.

The everyday version is simpler.

Observability is the difference between 'the website is slow' and 'the website is slow because the user service is hitting a connection pool limit, which started ten minutes ago when the recommendations service began retrying failed requests, which started because the model cache invalidated after a deploy, which kicked off a thundering herd of cache misses.'

The first is information. The second is understanding. Getting from one to the other is what observability tools are built to support.

Most discussions break observability into three pillars. Metrics, the numerical measurements you collect at regular intervals. Logs, the text events your applications produce when something happens. Traces, the records

of how a single request moved across multiple services. You need all three to understand what's happening inside a modern data centre.

Any one of them on its own gives you a partial picture at best.

Why a single server with *htop* won't cut it anymore

For a long time, the way you knew your system was healthy was by SSHing into it and running tools like *top*, *htop*, *vmstat*, or *iostat*. You looked at the screen. You saw CPU at fifty percent. You shrugged and moved on.

That model dies the moment you have more than a handful of machines. And modern data centres have thousands. Tens of thousands. Some of them, hundreds of thousands. You cannot SSH into each one. You cannot eyeball each terminal. You need a way to collect signals from everywhere and bring them to one place where humans and automation can both look at them.

That's the basic shape of every observability stack. Agents on every machine and inside every service, pushing or pulling data to a central system. Storage that can handle billions of data points over time. Query languages that let you slice the data in useful ways. Dashboards and alerts on top of all of it.

The tools have specific names.

Prometheus and VictoriaMetrics for metrics. Loki, Elastic, and OpenSearch for logs. Jaeger, Tempo, and Zipkin for traces. Grafana for visualisation. OpenTelemetry is the increasingly common standard for emitting all three kinds of data from your applications. We'll touch on some of these as we go, but the names matter less than understanding why the categories exist.

Walking through a typical incident

Let me sketch a typical incident to make this concrete.

Tuesday evening, around eight. Traffic to a streaming service is high but not unusually so. Suddenly the on-call engineer's phone buzzes. An alert. Page load latency in one region has crossed the five-second mark, which is the threshold for waking somebody up.

The engineer opens the laptop, navigates to the main Grafana dashboard, and sees what's going on. P95 latency is climbing steadily across all services in that region. CPU and memory look normal. So it's probably not a resource issue.

They click through to the service-by-service breakdown. Most services are fine. But the recommendations service is showing elevated error rates



Figure 1: Incident walk through